



Agentic Coding with Antigravity CLI

Google's AI Agent in Your Terminal

Press Space for next page →

Contact Info

Ken Kousen Kousen IT, Inc.

- ken.kousen@kousenit.com
- <http://www.kousenit.com>
- <http://kousenit.org> (blog)
- Social Media:
 - [@kenkousen](https://twitter.com/kenkousen) (twitter)
 - [@kenkousen@foojay.social](https://mastodon.social/@kenkousen) (mastodon)
 - [@kousenit.com](https://kousenit.com) (bluesky)
- *Tales from the jar side* (free newsletter)
 - <https://kenkousen.substack.com>
 - <https://youtube.com/@talesfromthejarside>

Course Overview

- **Duration:** 5 hours of hands-on learning
- **Format:** Instructor-led with multiple labs
- **Hands-on Labs:** Real codebases in Python, JavaScript, Java
- **Prerequisites:** Command-line experience, development background

Topics Covered

- **Foundation:** Installation, CLI basics, authentication
- **Core Skills:** File operations, shell integration, context management
- **Customization:** AGENTS.md, custom commands, settings.json
- **Safety:** Sandbox mode, permission rules, checkpointing
- **Advanced:** MCP integration, skills/plugins, session management

What's New in Antigravity CLI 1.0.x

- **Initial 1.0.0 release** of the Antigravity CLI — a Go-based terminal agent
- **Multi-model** sessions: Gemini, Claude, and GPT-OSS via `/model` (1.0.5)
- `models` **subcommand** + `--model` flag to pick a model at launch (1.0.5)
- `/permissions` command to manage tool permission rules in-CLI (1.0.5)
- **G1 credits** with `/credits` , `/usage` , `/quota` when quota runs out (1.0.3)
- **MCP via** `url` in `mcp_config.json` ; parallel server init (1.0.5/1.0.6)
- **SQLite conversation format** + `/resume` , `--continue` , `--conversation` (1.0.4)
- **Plugin discovery** for skills and subagents (1.0.1)
- **Latest stable track:** Antigravity CLI `1.0.6`

What is Antigravity CLI?

- Google's Go-based AI coding agent for the terminal (binary: `agy`)
- **Multi-model:** Gemini 3.x, Claude Sonnet/Opus 4.6, GPT-OSS — switch with `/model`
- Built-in tools: web search, file ops, shell, web fetch
- Model Context Protocol (MCP) support via `mcp_config.json`
- Shares config with the Antigravity 2.0 desktop app (`/export`)

Key Differentiators

- **Model choice:** mix Gemini, Claude, and GPT-OSS in one tool
- **Google Search Grounding:** real-time web access
- **Free tier + G1 credits:** keep working when quota runs out
- **Desktop continuity:** shares conversations with Antigravity 2.0
- **MCP Native:** built-in Model Context Protocol support

Models Available

- **Gemini 3.1 Pro:** high-capability Gemini model for complex coding
- **Gemini 3.5 Flash:** faster, lower-cost option (Low/Medium/High)
- **Claude Sonnet 4.6 / Opus 4.6:** Anthropic models (thinking)
- **GPT-OSS 120B:** open-weights option
- **List models:** `agy models` · **Switch:** `/model` or `--model`

Quota & Credits

- **Free tier:** sign in with a Google account to get started
- **G1 credits:** kick in automatically when standard quota runs out
- **In-CLI panels:** `/credits` , `/usage` , `/quota` for real-time status
- Manage models and preferences via `/settings`

📖 **Docs:** antigravity.google/docs

Installation

- **macOS / Linux:** `curl -fsSL https://antigravity.google/cli/install.sh | bash`
- **Windows (PowerShell):** `irm https://antigravity.google/cli/install.ps1 | iex`
- Installs the `agy` binary to `~/.local/bin/`
- Verify: `agy --version` · Update later: `agy update`

```
1  # Install (macOS / Linux)
2  curl -fsSL https://antigravity.google/cli/install.sh | bash
3
4  # Verify installation
5  agy --version
```

Authentication

- **Google Sign-In** (default): launches automatically on first run
- **Remote / SSH**: shows an authorization URL with a one-time code
- **API key (alternative)**: `export ANTIGRAVITY_API_KEY="your-key"`
- Credentials persist via OAuth in `~/.gemini/`

```
1  # First run: sign in with your Google account
2  agy
3
4  # Alternative: API key for scripts / headless use
5  export ANTIGRAVITY_API_KEY="your-api-key"
6  agy -p "Summarize this repo"
```

Basic Usage Modes

- **Interactive REPL:** `agy` - Start a conversation
- **One-shot / print:** `agy -p "prompt"` - Single response
- **Piped input:** `echo "task" | agy -p`
- **Interactive with initial prompt:** `agy -i "initial context"`

```
1  # Interactive mode
2  agy
3
4  # One-shot (print) mode
5  agy -p "Explain what this codebase does"
6
7  # Start interactively with an initial prompt
8  agy -i "You are a Python expert"
```

Core Features

Essential Capabilities

Master the fundamentals



File References with @

- Reference files directly: `@./src/main.js`
- Reference directories: `@./src/` (recursive)
- Reference images: `@./screenshot.png`
- Multiple references in one prompt

```
1  # Reference a specific file
2  agy -p "Explain @./src/app.py"
3
4  # Reference multiple files
5  agy -p "Compare @./old.js and @./new.js"
6
7  # Reference a directory
8  agy -p "Analyze the architecture in @./src/"
```

Shell Integration with !

- Execute shell commands: `!git status`
- Toggle persistent shell mode: `!`
- The agent can observe and analyze output
- Combine with AI analysis

```
1  # In interactive mode:
2  > !npm test
3  # The agent sees the test output
4
5  > !git diff
6  # Ask the agent to analyze the changes
7
8  # Toggle persistent shell mode
9  > !
```

Slash Commands: Navigation

- `/help` - Show available commands and shortcuts
- `/context` - View loaded context and token usage
- `/model` - Switch the active model mid-session
- `/settings` - Open settings and preferences

Slash Commands: Workflow

- `/agent <task>` - Dispatch an asynchronous subagent
- `/permissions` - Add/edit/remove tool permission rules
- `/usage` - Session, quota, and rate-limit status

Slash Commands: Session Control

- `/resume` - Open the conversation browser
- `/export` - Send the session to the Antigravity 2.0 desktop app
- `/credits` · `/quota` - Credit balance and quota panels

Slash Commands in Action (Context)

```
1  # Show what context is loaded and token usage
2  /context
3
4  # Switch the active model
5  /model
6
7  # Manage tool permission rules
8  /permissions
9
10 # Open settings and preferences
11 /settings
```

Slash Commands in Action (Sessions)

```
1  # Open the conversation browser
2  /resume
3
4  # Dispatch an async subagent task
5  /agent refactor the auth module
6
7  # Push this session to the desktop app
8  /export
```

Keyboard Shortcuts: Editing

- `Ctrl+L` - Clear screen
- `Ctrl+V` - Paste text/images
- `Ctrl+R` - Open the Artifact Review panel

Keyboard Shortcuts: Control

- `Esc` - Interrupt the active agent stream
- `Ctrl+C` - Cancel current operation
- `Ctrl+D` `Ctrl+D` - Exit Antigravity CLI (press twice)

Built-in Tools

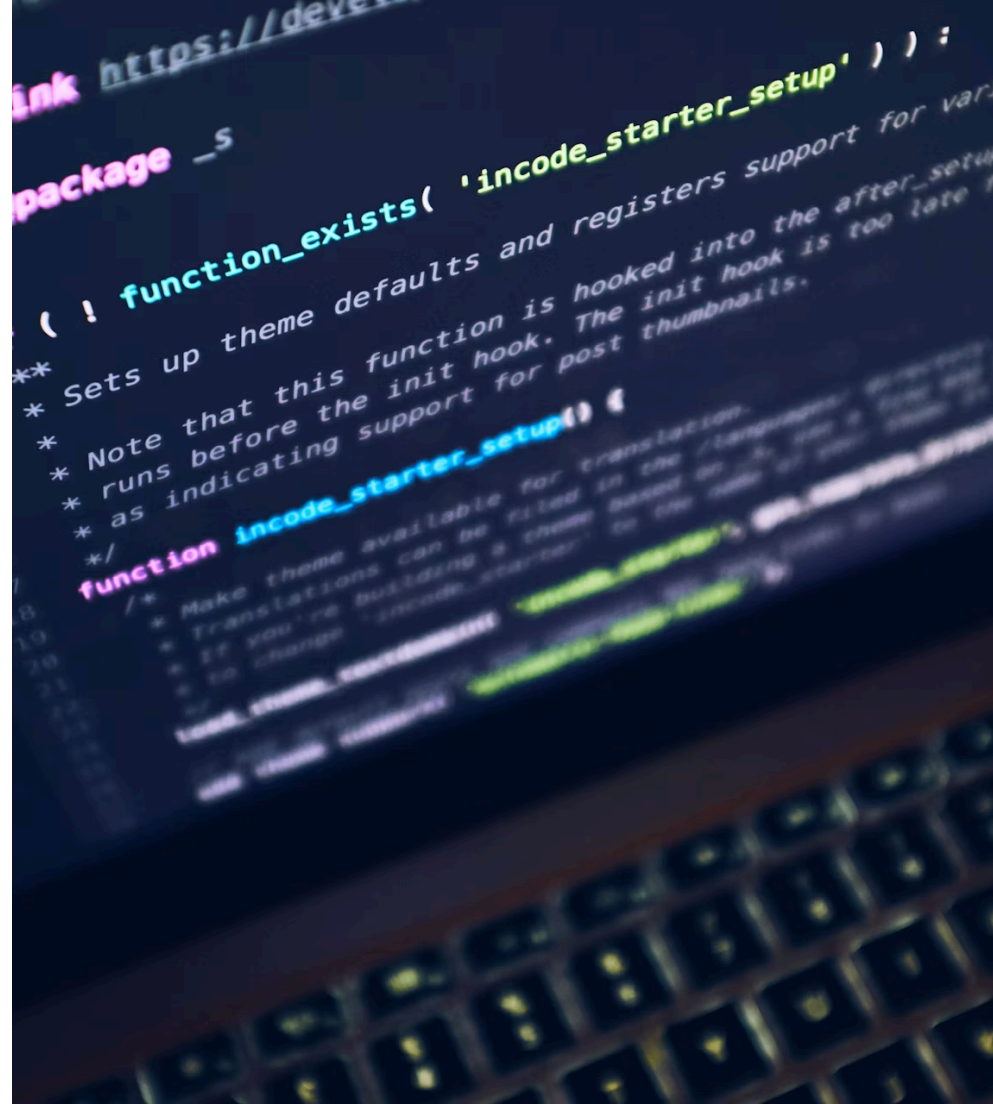
- **File System:** `read_file()`, `write_file()`, `replace()`, `glob()`
- **Shell:** Execute terminal commands
- **Web:** `google_web_search()`, `web_fetch()`
- **Memory:** `save_memory()` for cross-session recall

```
1  # The agent automatically uses appropriate tools
2  "Search the web for React 19 new features"
3  # Uses google_web_search()
4
5  "Read all Python files in src/"
6  # Uses glob() and read_file()
7
8  "Update the README with the changes we made"
9  # Uses write_file()
```

Safety & Control

Work Safely

Permission rules and sandboxing



Tool Permissions

- **Default:** prompt for approval on each tool call
- `/permissions` : add, edit, or remove allow/deny rules in-CLI
- `proceed-in-sandbox` : auto-approve commands that stay in the sandbox
- `--dangerously-skip-permissions` : auto-approve everything (use with care)

```
1  # Default - interactive, prompts per tool call
2  agy
3
4  # Auto-approve all tool calls (use with care)
5  agy --dangerously-skip-permissions
6
7  # Manage allow/deny rules from inside a session
8  > /permissions
```

Artifact Review

- Review proposed changes before they are applied
- Open the **Artifact Review panel** with `Ctrl+R`
- Inspect diffs with `/diff` (supports commit-hash selection)
- Works even while answering pending tool-permission prompts
- Great safety habit when exploring an unfamiliar codebase

```
1  # Inspect changes from inside a session
2  > /diff
3
4  # Toggle the Artifact Review panel
5  Ctrl+R
```

Sandbox Mode

- Isolate file operations away from your host
- Multiple backends depending on your OS
- Prevents accidental system changes
- Perfect for exploring unfamiliar code

```
1  # Run in sandbox mode (terminal restrictions enabled)
2  agy --sandbox
3
4  # Combine with a one-shot prompt
5  agy --sandbox -p "Refactor this entire codebase"
```

Sandbox & Permission Modes

- `--sandbox` : run with terminal restrictions enabled
- `proceed-in-sandbox` permission mode: auto-approve commands that stay inside the sandbox, prompt only when one tries to break out
- Sandbox isolation is enforced in headless print mode too (`-p`)
- Pair with `/permissions` rules for durable guardrails

```
1  # Interactive, sandboxed
2  agy --sandbox
3
4  # Non-interactive, sandbox still enforced
5  agy --sandbox -p "Audit @./src for risky calls"
```

 antigravity.google/docs

Checkpointing

- **Snapshots:** capture state before file modifications
- **Shadow storage:** kept under `~/.gemini/` (not your repo)
- **Includes:** files + conversation + tool call
- **Manage** checkpoints from the `/context` panel

```
1  # View context and manage checkpoints
2  > /context
```

Restoring Checkpoints

```
1  # Open the context panel to review checkpoints
2  /context
3
4  # Resume an earlier conversation by ID
5  agy --conversation <id>
6
7  # Continue the most recent conversation
8  agy -c
```

Recover earlier state and pick up where you left off



Context Management

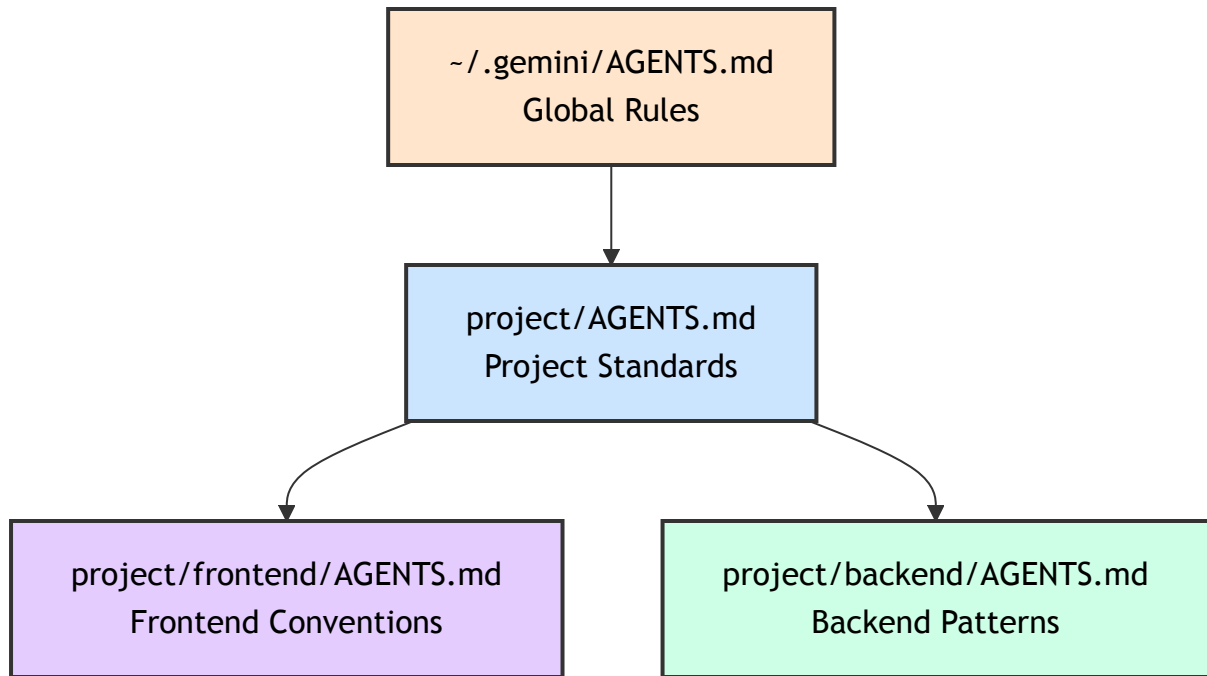
AGENTS.md Files

Project memory and instructions

What is AGENTS.md?

- **Project memory** loaded automatically
- **Coding standards** and conventions
- **Architecture context** for the AI
- **Persistent instructions** across sessions
- Antigravity also recognizes `GEMINI.md` and `CLAUDE.md`

Hierarchical Loading



More specific files override general ones

Example AGENTS.md

```
1  # Project: Weather API
2  ## Tech Stack
3  - Backend: Python Flask
4  - Database: PostgreSQL
5  - Testing: pytest
6  ## Coding Standards
7  - Use type hints for all functions
8  - Follow PEP 8 style guide
9  - Write docstrings for public APIs
10 ## Current Focus
11 Implementing caching layer for API responses
```

Managing Context

- Edit `AGENTS.md` directly to update project memory
- `/context` - view combined context and token usage
- Add `includeDirectories` in settings to load shared context
- Restart or reload the session to pick up changes

```
1  # Create or edit project memory
2  $EDITOR AGENTS.md
3
4  # See what context is loaded
5  > /context
```

Modular Imports

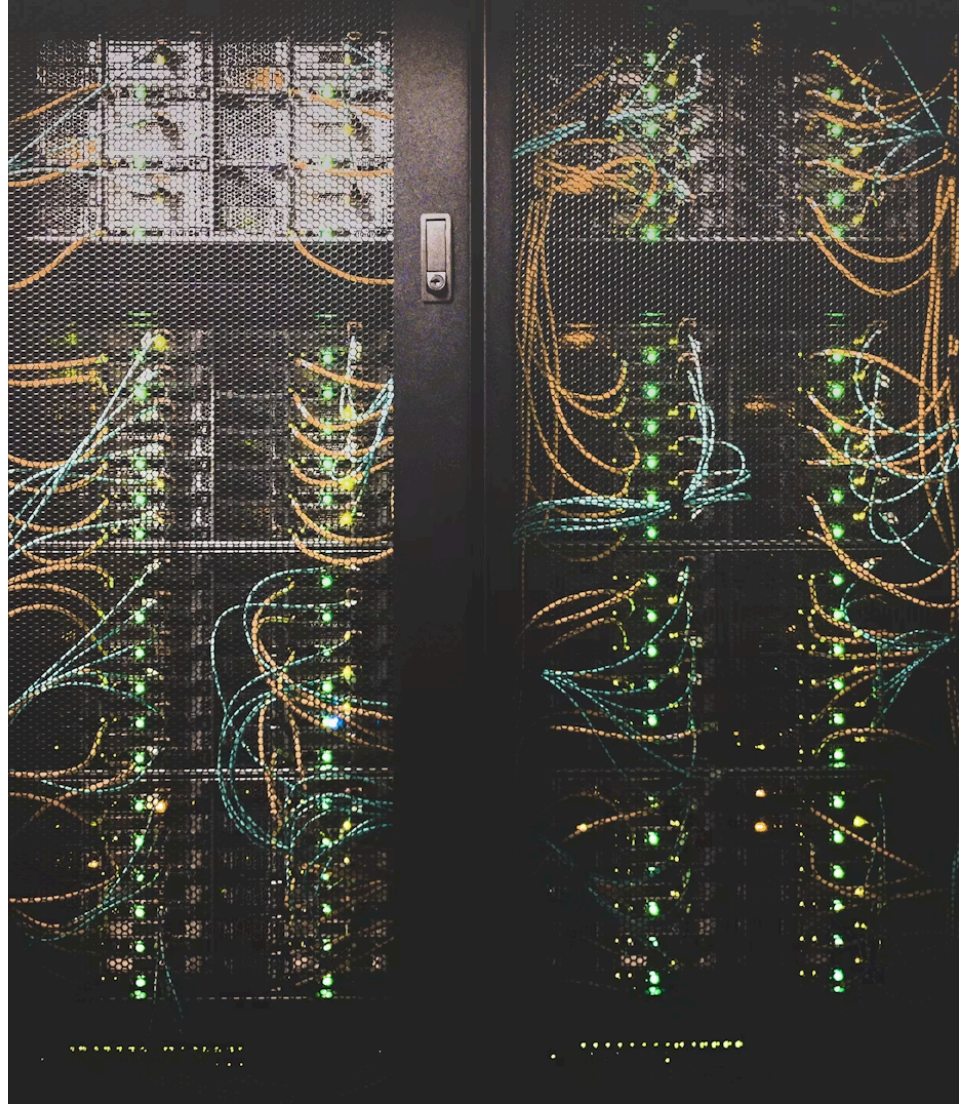
- Import other files with `@file.md` syntax
- Break large context into components
- Supports relative and absolute paths

```
1  # AGENTS.md
2
3  ## Project Overview
4  @./docs/architecture.md
5
6  ## Coding Standards
7  @./docs/style-guide.md
8
9  ## API Documentation
10 @./docs/api-reference.md
```

Configuration

Customize Your Setup

settings.json and environment



Configuration Layers

1. **Default values** - Built-in defaults
2. **User settings** - `~/.gemini/settings.json`
3. **Project settings** - `.gemini/settings.json`
4. **Environment variables** - Including `.env` files
5. **Command-line arguments** - Highest priority

settings.json Options

```
1  {  
2    "general": {  
3      "preferredEditor": "vscode",  
4      "vimMode": false,  
5      "checkpointing": { "enabled": true }  
6    }  
7  }
```

settings.json Options (UI + Tools)

```
1  {  
2    "ui": {  
3      "hideTips": false,  
4      "hideBanner": false  
5    },  
6    "tools": {  
7      "sandbox": false  
8    }  
9  }
```

Tool Permissions & Rules

- `/permissions` : add, edit, or remove allow/deny rules in-CLI
- Rules merge across three layers: project, user, and CLI settings
- Shared with the Antigravity desktop app's permission settings
- Keep high-risk tools constrained for teams

```
1  # Manage permission rules interactively
2  > /permissions
3
4  # Auto-approve everything (use with care)
5  agy --dangerously-skip-permissions
```

 antigravity.google/docs

File Filtering

- **respectGitIgnore:** Honor .gitignore patterns
- **enableRecursiveFileSearch:** Recursive completion
- Exclude rules and allowlists live in `rules.json`

```
1  {  
2    "context": {  
3      "fileFiltering": {  
4        "respectGitIgnore": true,  
5        "enableRecursiveFileSearch": true  
6      }  
7    }  
8  }
```

Environment Variables

Variable	Purpose
ANTIGRAVITY_API_KEY	API-key authentication (alternative to sign-in)
AGY_CLI_DISABLE_LATEX	Turn off LaTeX math rendering
AGY_CLI_HIDE_ACCOUNT_INFO	Hide email and plan tier from the header
\$EDITOR	External editor for prompts and files
HTTP_PROXY	Network proxy

Custom Context Filenames

- `AGENTS.md` is the documented default context file
- `GEMINI.md` and `CLAUDE.md` are also recognized
- Support multiple filenames in priority order
- Include additional directories for shared context

```
1  {
2    "context": {
3      "fileName": ["AGENTS.md", "GEMINI.md", "CLAUDE.md"],
4      "includeDirectories": ["~/shared-context"],
5      "loadMemoryFromIncludeDirectories": true
6    }
7  }
```

Advanced Features

Power User Tools

MCP, Plugins, Sessions



Model Context Protocol (MCP)

- Standard protocol for AI-to-system connections
- Connect to external tools and services
- Supports local commands, HTTP, and SSE
- OAuth 2.0 for remote authentication

MCP Configuration: Firecrawl

```
1  {  
2    "mcpServers": {  
3      "firecrawl": {  
4        "command": "npx",  
5        "args": ["@modelcontextprotocol/server-firecrawl"],  
6        "env": {  
7          "FIRECRAWL_API_KEY": "${FIRECRAWL_API_KEY}"  
8        }  
9      }  
10   }  
11 }
```

MCP Configuration: Database

```
1  {
2    "mcpServers": {
3      "postgres": {
4        "command": "npx",
5        "args": ["@modelcontextprotocol/server-postgres"],
6        "env": {
7          "CONNECTION_STRING": "${DATABASE_URL}"
8        }
9      }
10   }
11 }
```

Managing MCP Servers

Antigravity has no `mcp` subcommand — edit the config file directly:

```
1  # Open the MCP config
2  $EDITOR ~/.gemini/config/mcp_config.json

1  {
2      "mcpServers": {
3          "github": {
4              "command": "npx",
5              "args": ["-y", "@modelcontextprotocol/server-github"]
6          }
7      }
8  }
```

Servers initialize in parallel on startup.

MCP: Remote Servers

Use a `url` for HTTP/remote MCP servers (added in 1.0.5):

```
1  {  
2    "mcpServers": {  
3      "remote-tools": {  
4        "url": "https://mcp.example.com/sse"  
5      }  
6    }  
7  }
```

Toggle servers from the in-CLI settings; changes reload on restart.

MCP Server Options

- **command**: Shell command to start server
- **args**: Command arguments
- **env**: Environment variables
- **cwd**: Working directory

MCP Server Options (Advanced)

- **timeout**: Startup timeout in ms
- **includeTools/excludeTools**: Filter available tools
- **trust**: Bypass tool confirmation for this server

Popular MCP Servers

- **Firecrawl** - Web scraping, search, content extraction
- **PostgreSQL** - Database queries and schema
- **Filesystem** - Extended file operations
- **Slack** - Team communication
- **Playwright** - Browser automation

📖 **Registry:** modelcontextprotocol.io/registry

Plugins from Marketplaces

- Install from a marketplace with `plugin@marketplace` syntax
- Plugins bundle **skills** and **subagents**, discovered automatically
- Import existing Gemini or Claude plugins with `agy plugin import`
- Plugins install to the shared `~/.gemini/config/` directory

```
1 # Install, then link a marketplace
2 agy plugin install my-plugin@my-marketplace
3 agy plugin link my-marketplace ./target
```

 antigravity.google/docs

Subagents

- Dispatch focused tasks to a **subagent** with `/agent <task>`
- Subagents run with their own context and interaction timeout
- Bundled and discovered through installed plugins
- Standalone subagent conversations stay separate in `/resume`

```
1  # Dispatch an async subagent
2  > /agent write unit tests for @./src/utils.py
```

Plugin System

- Extend Antigravity CLI with **plugins** (skills + subagents)
- Managed with the `agy plugin` subcommand
- Import from Gemini or Claude: `agy plugin import gemini`
- Installed plugins are scanned for skills and agents automatically

```
1  # List installed plugins
2  agy plugin list
3
4  # Install / enable / disable
5  agy plugin install my-plugin@marketplace
6  agy plugin enable my-plugin
7  agy plugin disable my-plugin
```

Skills

- Package reusable expert workflows as **skills**
- Distributed inside plugins, stored under `~/.gemini/config/skills/`
- Surface as slash commands via autocomplete
- Discovered automatically from installed plugins

```
1  # Skills arrive with plugins
2  agy plugin install code-review@marketplace
3
4  # Invoke a skill-derived command in-session
5  > /code-review @./src
```

Session Management

- **Continue:** pick up the most recent conversation
- **Resume by ID:** jump to a specific past conversation
- **Browse:** open the conversation picker with `/resume`
- Conversations are stored in SQLite (`.db`)

```
1  # Continue the most recent conversation
2  agy -c
3
4  # Resume a specific conversation by ID
5  agy --conversation <id>
6
7  # Browse conversations from inside a session
8  > /resume
```

Non-Interactive (Print) Mode

- `-p` / `--print` : run a single prompt and print the response
- `--print-timeout` : bound how long print mode waits (default 5m)
- Sandbox isolation is enforced in print mode too
- Pipe input from other commands for scripting

```
1  # One-shot print mode
2  agy -p "List all TODO comments in @./src"
3
4  # Piped input
5  cat error.log | agy -p "Diagnose this stack trace"
```

Desktop Integration

- **Antigravity 2.0 desktop app:** shares config and conversations
- `/export` : push a CLI session into the desktop app
- **Shared permissions:** rules carry across CLI and desktop
- **External editor:** set `$EDITOR` for prompt and file editing

```
1  # Push this session to the desktop app
2  > /export
```



Practical Applications

Real-World Workflows

Common use cases

Code Exploration

- Understand unfamiliar codebases
- Trace dependencies and data flow
- Find patterns and conventions
- Generate architecture documentation

```
1 agy -p "Analyze the architecture of @./src/ and explain  
2 how the components interact"  
3  
4 agy -p "Trace the flow from the API endpoint to the database  
5 in @./src/controllers/ and @./src/services/"
```

Test Generation

- Generate unit tests for existing code
- Identify edge cases automatically
- Create integration test scaffolding
- Mock setup and fixtures

```
1 agy -p "Create comprehensive unit tests for @./src/utils.py
2 with pytest, including edge cases"
3
4 agy -p "Generate integration tests for @./src/api/users.py
5 with proper mocking"
```

Documentation Generation

- README files for projects
- API documentation
- Architecture diagrams (Mermaid)
- Code comments and docstrings

```
1 agy -p "Generate a comprehensive README.md for this project"
2
3 agy -p "Add detailed docstrings to all public functions
4 in @./src/services/"
5
6 agy -p "Create a Mermaid diagram showing the system architecture"
```

Refactoring & Modernization

- Upgrade legacy code patterns
- Apply modern language features
- Improve code organization
- Fix anti-patterns

```
1 agy -p "Refactor @./src/legacy.py to use modern Python 3.12  
2 features like type hints and match statements"  
3  
4 agy -p "Convert this callback-based code to async/await  
5 @./src/api.js"
```

Debugging Workflows

- Analyze error messages and stack traces
- Identify root causes
- Suggest fixes with context
- Test and verify solutions

```
1  agy -p "This test is failing with @./tests/output.log.  
2  Analyze the error and fix the issue in @./src/app.py"  
3  
4  agy -p "Debug why the API returns 500 errors.  
5  Check @./src/routes.py and @./src/middleware.py"
```

Git Workflows

- Generate commit messages
- Create pull request descriptions
- Analyze diffs and changes
- Resolve merge conflicts

```
1  # Analyze staged changes
2  !git diff --staged
3  "Generate a conventional commit message for these changes"
4
5  # Create PR description
6  "Create a pull request description summarizing the changes
7  from the last 5 commits"
```

CI/CD Integration

- Non-interactive `--print` mode for pipelines
- Capture stdout to a file for parsing
- Exit codes for success/failure
- Automated code reviews

```
1  # In CI/CD pipeline
2  agy -p "Review @./src/ for security issues" > review.txt
3
4  # Check exit code
5  if agy -p "Verify all tests pass"; then
6      echo "All checks passed"
7  fi
```

Optional Advanced Module

- Designed for teams moving from personal usage to production workflows
- Can be taught live or assigned as follow-up practice
- Focus areas: auth strategy, governance, skills/MCP, CI automation



Authentication Matrix (Team Use)

- **Google Sign-In:** best default for local interactive usage
- **Device-code flow:** shows a URL + code for remote/SSH machines
- `ANTIGRAVITY_API_KEY` : scripts and headless automation
- **G1 credits:** keep teams productive past standard quota

Auth Quick Checks

```
1  # Interactive route: sign in with Google on first run
2  agy
3
4  # API-key route (scripts / headless)
5  export ANTIGRAVITY_API_KEY="..."
6  agy -p "Summarize @./src"
7
8  # Hide account info from the header (e.g. for screen sharing)
9  export AGY_CLI_HIDE_ACCOUNT_INFO=1
10 agy
```

Governance: Permissions and Rules

- `--dangerously-skip-permissions` is a per-session escape hatch
- `/permissions` rules provide durable guardrails across users/projects
- Keep high-risk tools constrained in team settings
- Prefer explicit allow/deny patterns over ad-hoc approvals

Governance: Folder Trust

- Antigravity tracks trusted folders (`trustedFolders.json`)
- Folder trust impacts settings, skills, and context loading
- Treat untrusted repos with `--sandbox` and stricter rules
- Combine with `/permissions` for auditable guardrails

Skills + MCP at Scale

- Skills package repeatable expert workflows for teams
- MCP connects external systems (docs, data, browsers, APIs)
- Distribute skills and subagents together inside plugins
- Use `includeTools` / `excludeTools` to narrow risky MCP exposure

Plugins + MCP Commands

```
1  # Plugin (skills + subagents) lifecycle
2  agy plugin list
3  agy plugin disable my-plugin
4  agy plugin enable my-plugin
5
6  # MCP servers: edit config, then restart to reload
7  $EDITOR ~/.gemini/config/mcp_config.json
```

Automation Patterns (Non-Interactive)

- Prefer `--print` for non-interactive runs
- Use exit codes for pipeline gates
- Keep prompts deterministic and repo-scoped
- Store logs/artifacts for auditability

CI Example: Gate + Report

```
1  agy -p "Review @./src/ for security issues" > review.txt
2
3  if agy -p "Run checks and report pass/fail"; then
4      echo "Gate passed"
5  else
6      echo "Gate failed" && exit 1
7  fi
```

Best Practices

Professional Workflows

Tips for success



Effective Prompting

- **Be specific** about what you want to achieve
- **Provide context** about goals and constraints
- **Use file references** to ground the conversation
- **Iterate** for complex tasks
- **Include examples** when possible

Safety Guidelines

- **Start in default mode** - Get comfortable first
- **Enable checkpointing** - Safety net for mistakes
- **Review generated code** - Don't blindly accept
- **Use sandbox** for exploratory work
- **Commit often** - Git is your safety net

Project Setup

1. Create a comprehensive AGENTS.md
2. Set up project-specific settings.json
3. Configure relevant MCP servers
4. Create custom commands for common tasks
5. Establish team conventions

AGENTS.md Best Practices

- **Keep it focused** - Relevant project info only
- **Update regularly** - Reflect current state
- **Use imports** - Modularize large contexts
- **Include examples** - Show expected patterns
- **Document conventions** - Style guides, patterns

Team Collaboration

- Share AGENTS.md in version control
- Standardize settings.json across team
- Create shared custom commands
- Document AI-assisted workflows
- Review AI-generated code together

Common Pitfalls

- **Overly broad prompts** → Be specific
- **Missing context** → Use AGENTS.md
- **Skipping review** → Always verify output
- **Skipping permissions** → Don't reach for `--dangerously-skip-permissions` early
- **Ignoring Artifact Review** → Inspect diffs before applying

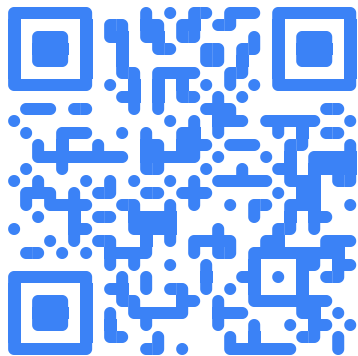
Troubleshooting

- **Authentication issues:** Re-run `agy` to sign in, or check `ANTIGRAVITY_API_KEY`
- **Rate limits:** Check `/quota` ; G1 credits cover overflow
- **Tool failures:** Verify `mcp_config.json` and server status
- **Context not loading:** Confirm `AGENTS.md` location, reload session
- **Logs:** Override the log path with `--log-file`

```
1  # Write logs to a known path for troubleshooting
2  agy --log-file ./agy.log
3
4  # Inspect loaded context and token usage
5  > /context
```

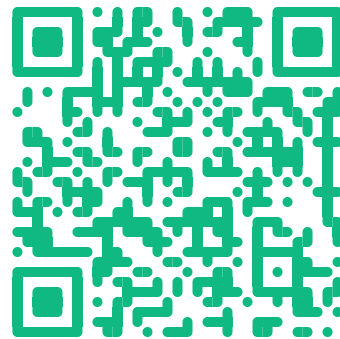
Quick Access

Antigravity CLI Docs



antigravity.google/docs

Course Repository



github.com/kousen/gemini-training

Important Links



Official Documentation

<https://antigravity.google/docs>



Source Repository

<https://github.com/google-antigravity/antigravity-cli>



MCP Server Registry

<https://modelcontextprotocol.io/registry>



Course Materials

<https://github.com/kousen/gemini-training>

Command Reference: Basic Usage

```
1  # Interactive mode
2  agy
3
4  # One-shot (print) mode
5  agy -p "prompt"
6
7  # Interactive with initial prompt
8  agy -i "context"
```

Command Reference: Safety

```
1  # Run in sandbox mode
2  agy --sandbox
3
4  # Auto-approve all tool calls (use with care)
5  agy --dangerously-skip-permissions
6
7  # Manage permission rules in-session
8  > /permissions
```

Command Reference: Sessions

```
1  # Continue the most recent conversation
2  agy -c
3
4  # Resume a conversation by ID
5  agy --conversation <id>
6
7  # Browse conversations in-session
8  > /resume
```

Command Reference: Output

```
1  # Print mode for scripting
2  agy -p "prompt"
3
4  # Bound how long print mode waits
5  agy --print-timeout 2m -p "prompt"
6
7  # Write logs to a file
8  agy --log-file ./agy.log
```

Thank You!

Questions?



Kenneth Kousen *Author, Speaker, Java & AI Expert*

kousenit.com | [@kenkousen](https://twitter.com/kenkousen)